RegisterPage Test Report

We are using Jest to test our code.

Test 1: Fills out the register form and correctly triggers the register request

Objective: Verify that the register form can be filled out correctly by the user and triggers the register request using useForm.post with the correct data.

Steps:

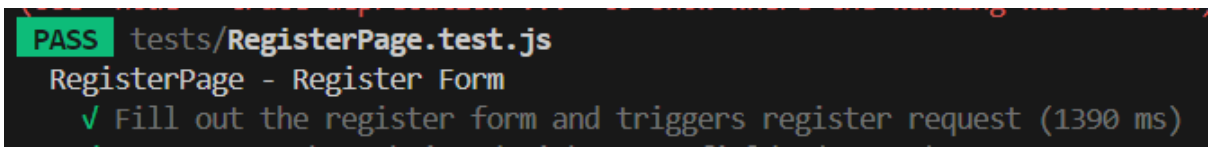
- Render the RegisterPage.
- Fill out the inputs
    - First Name: John
    - Last Name: Doe
    - Email: john.doe@example.co.uk
    - Password: Password123!
    - Confirm Password: Password123!
    - Phone Number: 07512345678
    - Allergies: No
- Click the register button.
- Verify that useForm.post is called correctly with the correct data.

```
test('Fill out the register form and triggers register request', async () => {
  const mockPost = jest.fn();
  useForm.mockReturnValue({
    data: {
        firstName: 'John',
        lastName: 'Doe',
        email: 'john.doe@example.co.uk',
        password: 'Password123!',
        confirmPassword: 'Password123!',
        phoneNumber: '07512345678',
        allergies: 'no',
    },
    setData: jest.fn(),
    post: mockPost,
    errors: {},
  });

  render(<RegisterPage/>);

  //Get inputs
  const firstNameInput = screen.getByLabelText(/first name/i);
  const lastNameInput = screen.getByLabelText(/last name/i);
  const emailInput = screen.getByTestId('email-input');
  const passInput = screen.getByTestId('password-input');
  const confirmPassInput = screen.getByTestId('confirm-password-input');
  const phoneInput = screen.getByLabelText(/phone number/i);
  const allergyInput = screen.getByRole('combobox', {name: /do you have allergies/i});
  const registerButton = screen.getByRole('button', {name: /register/i});
```

```
  //Fill out the form
  await userEvent.type(firstNameInput, 'John');
  await userEvent.type(lastNameInput, 'Doe');
  await userEvent.type(emailInput, 'john.doe@example.co.uk');
  await userEvent.type(passInput, 'Password123!');
  await userEvent.type(confirmPassInput, 'Password123!');
  await userEvent.type(phoneInput, '07512345678');
  await userEvent.selectOptions(allergyInput, 'no');

  //Check input for correct values
  expect(firstNameInput).toHaveValue('John');
  expect(lastNameInput).toHaveValue('Doe');
  expect(emailInput).toHaveValue('john.doe@example.co.uk');
  expect(passInput).toHaveValue('Password123!');
  expect(confirmPassInput).toHaveValue('Password123!');
  expect(phoneInput).toHaveValue('07512345678');
  expect(allergyInput).toHaveValue('no');

  //Simulate clicking register button
  await userEvent.click(registerButton);

  //Wait for the post function to be called with the correct data
  await waitFor(() => {
    expect(mockPost).toHaveBeenCalledWith('/post/registerUser', {
      firstName: 'John',
      lastName: 'Doe',
      email: 'john.doe@example.co.uk',
      password: 'Password123!',
      confirmPassword: 'Password123!',
      phoneNumber: '07512345678',
      allergies: 'no',
    })
  });
});
```

Findings:

- The form filled out correctly.
- useForm.post was successfully triggered with the expected payload sent to /post/registerUser.



Test 2: Form can not be submitted with empty fields

Objective: Ensure that the form does not submit when required fields are empty and that it displays the correct validation errors.

Steps:

- Render the RegisterPage.
- Click the register button without fill out the form.
- Verify that all the required fields are marked as invalid.
- Ensure that useForm.post is not called.

```
test('Form cannot be submitted with empty fields', async () => {
  const mockPost = jest.fn();
  useForm.mockReturnValue({
          data: {
            firstName: '',
            lastName: '',
            email:'',
            password: '',
            confirmPassword: '',
            phoneNumber: '',
            allergies: 'no',
            allergyInfo: ''
          },
          setData: jest.fn(),
          post: mockPost,
          errors: {},
      });

  render(<RegisterPage/>);

  //Get inputs
  const firstNameInput = screen.getByLabelText(/first name/i);
  const lastNameInput = screen.getByLabelText(/last name/i);
  const emailInput = screen.getByTestId('email-input');
  const passInput = screen.getByTestId('password-input');
  const confirmPassInput = screen.getByTestId('confirm-password-input');
  const phoneInput = screen.getByLabelText(/phone number/i);
  const allergyInput = screen.getByRole('combobox', { name: /do you have allergies/i });
  const registerButton = screen.getByRole('button', { name: /register/i });

  //Simulate clicking register button
  await userEvent.click(registerButton);

  //Wait for validation (HTML5 default browser validations)
  await waitFor(() => {
    expect(firstNameInput).toBeInvalid();
    expect(lastNameInput).toBeInvalid();
    expect(emailInput).toBeInvalid();
    expect(passInput).toBeInvalid();
    expect(confirmPassInput).toBeInvalid();
    expect(phoneInput).toBeInvalid();
  });

  //Ensure the post function was not called since the form was not submitted
  await waitFor(() => expect(mockPost).not.toHaveBeenCalled());
});
```

Findings:

- The form correctly marked the fields as invalid.
- No submission occurred and useForm.post was not called.